Chocolate bar cutting problem - Alex Keshavarzi

This repository contains my solution to the chocolate bar cutting problem. Given `m` starting chocolate bars and the requested amounts for `n` children, the aim is to distribute the requested chocolate using as few cuts as possible.

The main solution is `solve4.py`. It is an approximate solution intended to remain practical as the inputs grow. `solve5.py` finds the exact minimum and provides a reference solution for smaller cases, but has exponential time complexity. The earlier solver files have been retained to be as transparent as possible and show how the approach developed as I tested the problem and clarified its assumptions.

Contents

1. Problem interpretation

I explicitly checked the following assumptions and constraints with Jake:

- `start` is a list containing the sizes of the `m` starting chocolate bars.
- `target` is a list containing the amounts requested by the `n` children.
- All chocolate amounts are represented by positive integers.
- One cut splits one existing piece into exactly two new pieces.
- Both pieces created by a cut must have a size of at least one.
- Giving an existing whole piece to a child requires no cut.
- A child may receive several smaller pieces of chocolate.
- Those pieces may come from different starting bars.
- Not all available chocolate has to be used. A remainder is allowed, so `sum(target)` may be less than `sum(start)`.
- Ordering has no meaning. Neither input list is assumed to arrive sorted.
- Duplicate values are allowed in both lists. The children do not all request the same amount, but their individual requests do not have to be unique.
- A solution should not be assumed to exist. The program must handle cases where the children cannot all receive their requested amounts exactly.

For the final point, an exact solution is impossible only if `sum(target) > sum(start)`. I chose to handle this as follows:

- If `sum(target) > sum(start)`, the targets are reduced proportionally to the available chocolate while ensuring that every child receives at least one unit.

- The returned `exact_target` flag is set to `False`, and `target_used` records the adjusted allocation, so the caller can see that the original requests were not met exactly.
- If there is not even one unit available per child, a self-imposed "fairness" constraint cannot be met and `reduce_targets()` raises a `ValueError`.

The important clarification is that one child's allocation does not need to come from one piece or one starting bar. For example:

```
start = [3,3]
target = [6]
```

requires no cuts: both complete bars can be given to the same child. This clarification invalidated a restriction used in `solve1.py`, `solve2.py` and `solve3.py`, and motivated the current implementations.

2. Current approach

Approximate solver

`solve4.py` is my primary solution. It repeatedly tries to find the cheapest matches first in the following order:

1. Remove exact matches between a remaining bar and a remaining target. This requires no cut.
2. Match two complete bars to one child's remaining target. This also requires no cut.
3. Match one bar to two targets, including a possible dummy target representing unused chocolate. This requires one cut.
4. If none of these matches work:
    - take the largest complete bar smaller than the largest target and give it to that child; or
    - take the smallest bar larger than the target and cut off the exact amount needed.

This last step is a local rather than global optimisation. It does not check every possible effect on later choices. This makes the solution fast, avoiding exponential growth, but means it does not always find the minimum number of cuts.

The function returns a dictionary:

```
{
    "cuts": cuts,
    "exact_target": exact_target,
    "target_used": adjusted_target
}
```

`exact_target` records whether the original request could be met. If there was insufficient chocolate, `target_used` contains the proportionally reduced allocation used by the solver.

Exact reference solver

`solve5.py` searches the complete state space under the corrected assumptions. It uses a deque and treats allocations differently according to their cost:

- Giving away a complete piece costs no cuts, so the resulting state is placed at the front of the queue.
- Splitting a piece costs one cut, so the resulting state is placed at the back.
- A `best` dictionary records the smallest number of cuts used to reach each sorted state and prevents inferior repeated work.

This method finds the minimum number of cuts, but the number of possible states grows very quickly. It is useful for checking `solve4` on small problems, but it is not practical for running larger problems. It is not the preferred solution because the number of possible states grows exponentially with the input size.

3. Repository structure

```
TakeHomeTest/
|-- main.py              Runs examples and the full analysis
|-- solve4.py            Main fast approximate solution
|-- solve5.py            Exact solution for small cases
|-- helpers.py           Shared matching and target adjustment functions
|-- generate_truths.py   Creates random tests with known answers
|-- analysis.py          Runs trials and creates plots
|-- solve1.py            First attempt, retained for development history
|-- solve2.py            Improved search using the original wrong assumption
|-- solve3.py            Faster approach using the original wrong assumption
|-- AKeshavarzi_RiverlaneCodingChallenge-Notes_10-08-26.pdf
|                          Handwritten notes from developing the solution
`-- plots/               Saved analysis figures
```

4. File descriptions and connections

`main.py`

This is the entry point for running the code. It imports all five solver attempts, although only `solve4` and `solve5` are enabled in the `solutions` dictionary.

It contains a set of hand-written states covering:

- cases requiring no cuts;
- insufficient chocolate;
- one child receiving multiple complete pieces from different bars;
- the example supplied with the problem;
- exact use of all available chocolate;
- repeated target values; and
- larger inputs.

`exec_timer()` runs each enabled solver repeatedly, checks that it returns a consistent result and reports mean and maximum execution times.

After the hand-written checks, `main.py` runs two broader analyses:

- `solve4` alone over larger values of `m` and `n`; and
- `solve4` against `solve5` over smaller values, where the exact solver remains practical.

`helpers.py`

This contains the reusable operations needed by the current solvers:

- `remove_exact_matches()` removes bar/target pairs which already agree and therefore need no cuts.
- `two_sum()` finds two values which add up to one of a set of requested totals.
- `reduce_targets()` proportionally reduces requests when there is insufficient chocolate, while maintaining a minimum allocation of one unit per child. It uses the largest-remainder method to distribute units lost by rounding down.

`solve4.py` uses all three helpers. `solve5.py` only needs `reduce_targets()` because its state search handles matching directly.

generate_truths.py

To measure accuracy, I need test problems where I already know the true minimum number of cuts. `generate_truths.py` creates these in reverse: it creates the children's targets first and then combines them to make the starting bars.

For a test with `m` bars and `n` children, it works as follows:

1. Generate `n` random target amounts.
2. Create `m` empty groups.
3. Put one target into each group. This makes sure that no group is empty.
4. Place every remaining target into a randomly chosen group.
5. Add together the targets in each group. Each total becomes one starting bar.
6. Shuffle both lists so the solver cannot use the original grouping or ordering.

For example, the generated targets could be:

```
[2,3,4,5]
```

They could be split into two hidden groups:

```
[2,5] and [3,4]
```

These groups produce the starting bars:

```
[7,7]
```

The known construction is to split each starting bar back into its original targets. A group containing `k` targets needs `k - 1` cuts. Across all `m` groups, the total is:

```
(targets in group 1 - 1) + ... + (targets in group m - 1) = n - m
```

This proves that `n - m` cuts are enough.

It is also impossible to use fewer. The test starts with `m` pieces and must supply `n` positive target amounts. Each cut increases the number of pieces by exactly one, so creating at least `n` pieces from `m` starting pieces requires at least `n - m` cuts.

Therefore, `n - m` is both possible and the lowest possible value. This gives every generated trial a known true answer without needing to run the exact solver.

The generator requires `m <= n`. This is why an analysis with `max_n = 5` cannot contain cases above `m = 5`. If `m = n`, every group contains one target and the correct answer is zero cuts.

analysis.py

This module connects the random generator to one or more solver functions. For each `(m,n)` case it:

1. generates `n_tests` random problems with known answers;
2. records the solver output and execution time;
3. checks that the returned number of cuts is not below the known lower bound;
4. calculates accuracy, median time and mean time;
5. produces accuracy and timing plots against both `n` and `n/m`; and
6. plots the distribution of additional cuts used above the known minimum.

When given several solvers, `run_analysis()` creates separate figures for each solver and combined figures for direct comparison. Colours distinguish values of `m`, while line styles distinguish solver functions. Point markers ensure that a series containing only one valid case, such as `m = n = 5`, remains visible.

The plots show:

- accuracy against n;
- accuracy against n/m;
- median computation time against n;
- median computation time against n/m; and
- the number of extra cuts used when the minimum was not found.

The cut-difference histogram gives more information than the percentage accuracy alone. For each trial it calculates:

```
cuts found - known minimum cuts
```

A value of zero means that the solver found an optimal solution—these successful trials are excluded from the histogram. The remaining positive values show how many unnecessary cuts were used when the solver did not find the minimum. The y-axis shows the percentage of all trials.

Each histogram displays the maximum m, maximum n, number of trials per (m,n) case, number and percentage of non-optimal trials, and total number of trials represented. Individual histograms are produced for each solver, together with an overlaid comparison when more than one solver is analysed. Every histogram is saved twice: a linear y-axis version gives the clearest view of the most common errors, while a logarithmic version makes rare error sizes easier to see. If every trial is optimal, the plot states that there were no non-optimal trials.

Plot filenames include the solver name, maximum m, maximum n, number of tests, measurement type and axis variant. Generated figures are saved in plots/.

For a fair solver-to-solver comparison, pass a fixed seed. run_experiment() resets the random generator for each solver, so a shared seed makes them receive the same sequence of test problems. With seed=None, each solver receives a different random sample.

solve1.py

This is my initial breadth-first search (BFS) implementation. It tries every possible split of every remaining bar and therefore explores a very large number of states. It does not track visited states, uses an inefficient list as a queue and does not robustly handle branches with no solution. It is retained as a record of the first approach, but is not used by main.py.

solve2.py

This improves my first breadth-first search (BFS) by using a deque, tracking visited states, restricting candidate cuts to target values and pruning branches which cannot satisfy the remaining requests. Despite these improvements, its state space still grows exponentially. More importantly, it retains the original incorrect assumption that each child's amount must be obtained as a single piece.

solve3.py

This replaces the exponential search with my first approximate solution to avoid exponential time. Targets are processed from largest to smallest and assigned to the bar which leaves the smallest remainder. The resulting algorithm is much faster, but still suffers from the incorrect assumption that each child must receive one piece from one bar. It is retained as a record of staged approaches, but is not used by main.py.

solve4.py

See above.

solve5.py

See above.

`AKeshavarzi_RiverlaneCodingChallenge-Notes_10-08-26.pdf`

This PDF contains my handwritten notes from working through the problem and developing the solution. It is included to show my original reasoning and how the different approaches developed.

`plots/`

This directory contains generated PNG figures. The plots are outputs of `analysis.py`. They record accuracy, the distribution of additional cuts and execution time for individual solvers and for direct `solve4`/`solve5` comparisons.

5. Running the code

The code requires Python together with NumPy and Matplotlib. From the repository directory, run:

```
python main.py
```

The hand-written examples are printed first. The analysis then runs the configured random trials, displays the figures and saves them under `plots/`.

The exact solver can become slow very quickly. The comparison in `main.py` therefore limits both `max_m` and `max_n` to 5, while the larger analysis only uses `solve4`.

6. Using the solvers directly

```
from solve4 import solve4
from solve5 import solve5

start = [2,5,7]
target = [4,3,2,1]

approximate_result = solve4(start,target)
exact_result = solve5(start,target)
```

To analyse one solver:

```
from analysis import run_analysis
from solve4 import solve4

run_analysis(
    ("solve4",solve4),
    max_n=30,
    max_m=10,
    n_tests=1000,
    seed=1
)
```

To compare the two current solvers on the same generated cases:

```
from analysis import run_analysis
from solve4 import solve4
from solve5 import solve5

run_analysis(
    [("solve4",solve4),("solve5",solve5)],
    max_n=5,
    max_m=5,
    n_tests=10000,
    seed=1
)
```

7. Analysis findings

The generated tests measure the accuracy and computation time of `solve4`. Each test has a known minimum of `n - m` cuts.

Both analyses use `seed=1`. This makes the results repeatable and ensures that `solve4` and `solve5` receive the same problems when they are compared.

Accuracy results

The large `solve4` analysis tested up to 10 starting bars and 30 children. It ran 1,000 trials for every valid `(m,n)` pair, giving 255,000 trials in total.

Relevant plots: plots/solve4_max_m_10_max_n_30_n_tests_1000_accuracy_vs_n.png and plots/solve4_max_m_10_max_n_30_n_tests_1000_accuracy_vs_n_per_m.png.

`solve4` found the minimum in 237,118 trials, or 92.99%. Of the 17,882 trials which missed the minimum:

- 17,424 were one extra cut above the minimum;
- 452 were two extra cuts above the minimum; and
- 6 were three extra cuts above the minimum.

No trial was more than three cuts above the minimum. Of the trials which missed the minimum, 97.44% were only one cut away. Therefore, `solve4` did not always find the best answer, but almost every incorrect result was very close to it.

The cut-difference histograms show this distribution. Successful trials where the minimum was found are not shown. The y-axis still shows each group as a percentage of all trials. The information box gives the total number of trials and the number and percentage which missed the minimum. A logarithmic version is also produced so that the rare two-cut and three-cut errors remain visible.

Relevant plots: plots/solve4_max_m_10_max_n_30_n_tests_1000_accuracy_cut_difference_histogram_linear.png and plots/solve4_max_m_10_max_n_30_n_tests_1000_accuracy_cut_difference_histogram_log.png.

The accuracy of `solve4` depends strongly on the relationship between `m` and `n`. The accuracy plot against `n/m` shows the largest drop around `n/m = 1.5–2`.

When `n/m = 1`, every generated starting bar corresponds to one target, so the problem can be solved using exact matches. Just above this value, only some bars need to supply more than one target. There are fewer ways to group the values correctly.

If the direct matching steps do not find the correct grouping, the final approximation step may make a choice which appears useful at the time but prevents the best arrangement later. For example, it may give away a complete smaller bar or cut a larger bar without checking how that choice affects the remaining targets. Although not confirmed, this appears to be why the lack of look-ahead causes the most problems around `n/m = 1.5–2`.

Accuracy improves again as `n/m` increases. With more targets per starting bar, there are more possible ways to divide the available chocolate and still reach the known minimum.

For the smaller comparison, both `solve4` and `solve5` found the minimum in every test up to `m = n = 5`, using 10,000 trials for each valid `(m,n)` pair. However, I could not confirm the same result for `solve5` on larger cases because its computation time grew too quickly.

Relevant plots: plots/solve4_vs_solve5_max_m_5_max_n_5_n_tests_10000_combined_accuracy_vs_n.png and plots/solve4_vs_solve5_max_m_5_max_n_5_n_tests_10000_combined_accuracy_vs_n_per_m.png.

Time results

For the larger `solve4` analysis, median computation time increased smoothly as `n` increased and remained practical across the tested range.

Relevant plots: plots/solve4_max_m_10_max_n_30_n_tests_1000_time_median_vs_n.png and plots/solve4_max_m_10_max_n_30_n_tests_1000_time_median_vs_n_per_m.png.

Median time is used because individual timing measurements appeared to be affected occasionally by background activity or other one-off delays. These unusually slow measurements pulled the mean upwards. The median gave a better measure of the typical time taken by a trial. The mean is still calculated and stored.

For the small comparison, `solve5` was typically close to one order of magnitude slower than `solve4`. For the more difficult tested comparison cases, it was more than two orders of magnitude slower.

Relevant plots: plots/solve4_vs_solve5_max_m_5_max_n_5_n_tests_10000_combined_time_vs_n.png and plots/solve4_vs_solve5_max_m_5_max_n_5_n_tests_10000_combined_time_vs_n_per_m.png.

This difference follows from how the two solutions work. `solve4` makes one sequence of choices and continues until all targets have been handled. `solve5` checks many different possible arrangements before it can confirm which one uses the fewest cuts. The number of possible arrangements grows quickly as the problem becomes more complicated.

Running time is affected by both the size and structure of a problem. In the generated tests, when `m = n`, each starting bar is formed from exactly one target. These cases can be resolved through exact matches, which is why `(5,5)` can be easier than some cases with smaller `m` but more possible ways of assigning the chocolate.

Testing `solve5` beyond `m,n = 5` was not practical when running 10,000 trials for every pair. This does not mean that `solve5` cannot solve an individual larger problem. It means that running enough larger trials for a fair and useful comparison becomes too slow and unpredictable. For this reason, `solve5` is useful as an exact check for small cases, while `solve4` is the more practical choice for larger inputs.

Weaknesses, possible improvements and conclusions

The main weakness of `solve4` is that its final approximation step makes one local choice at a time without checking how that choice affects the rest of the problem. Once a bar has been assigned or cut, that choice cannot be changed later.

The direct matching steps also check matches involving only one or two values. They potentially miss a better arrangement involving three or more bars or targets.

The two main improvements I would investigate are:

- adding a limited look-ahead before choosing the next allocation; and
- extending the matching stage to check useful combinations of more than two values.

Both changes could improve accuracy, particularly around `n/m = 1.5–2`, but would have to be profiled in terms of the extra computation required.

The analyses reported here use a fixed random seed (`seed=1`) so that the results can be repeated and both solvers receive the same test problems. As a further check, I would repeat the analysis with several different seeds to confirm that the accuracy pattern, cut-difference distribution and computation-time comparison remain similar across different random samples.

However, overall the results support using `solve4` as the main solution. It is not guaranteed to find the minimum, but it remained practical across the larger test range and almost every incorrect result was only one cut above the minimum. `solve5` provides an exact comparison for small problems, but its running time grows too quickly for the larger analysis.